

IMSR-DB

MySQL Indexing DDL Reference
recommendation.py & app.py

All indexes are created via **ALTER TABLE ... ADD INDEX** guarded by an **information_schema.STATISTICS** check, making them safe to run on every startup and compatible with MySQL 5.6, 5.7, and 8.x (no IF NOT EXISTS syntax required).

Movie Table Indexes

Index Name	Table	Column(s)	Purpose
idx_movie_rating	movie	rating	Speeds up ORDER BY rating DESC on every startup load and trending/top-rated queries
idx_movie_year	movie	year	Speeds up ORDER BY year DESC for sort-by-year queries
idx_movie_title	movie	title	Enables fast prefix LIKE search on movie title

DDL Statements:

```
ALTER TABLE `movie`  
ADD INDEX `idx_movie_rating` (rating);  
ALTER TABLE `movie`  
ADD INDEX `idx_movie_year` (year);  
ALTER TABLE `movie`  
ADD INDEX `idx_movie_title` (title);
```

Relation / FK Table Indexes

Index Name	Table	Column(s)	Purpose
idx_movie_genre_mid	movie_genre	Movie_id	FK join in bulk load — avoids full table scan
idx_movie_genre_genre	movie_genre	genre	Supports genre-based filtering queries
idx_movie_director_mid	movie_director	Movie_id	FK join — avoids full table scan
idx_movie_director_name	movie_director	director_name	Enables fast director name search
idx_movie_cast_mid	movie_cast	Movie_id	FK join — avoids full table scan
idx_movie_cast_actor	movie_cast	actor_name	Enables fast actor name search
idx_movie_keywords_mid	movie_keywords	Movie_id	FK join — avoids full table scan
idx_movie_keywords_keyword	movie_keywords	keyword	Enables fast keyword search

DDL Statements:

```
ALTER TABLE `movie_genre`  
ADD INDEX `idx_movie_genre_mid` (Movie_id);  
ALTER TABLE `movie_genre`  
ADD INDEX `idx_movie_genre_genre` (genre);  
ALTER TABLE `movie_director`  
ADD INDEX `idx_movie_director_mid` (Movie_id);
```

```
ALTER TABLE `movie_director`
ADD INDEX `idx_movie_director_name` (director_name);
ALTER TABLE `movie_cast`
ADD INDEX `idx_movie_cast_mid` (Movie_id);
ALTER TABLE `movie_cast`
ADD INDEX `idx_movie_cast_actor` (actor_name);
ALTER TABLE `movie_keywords`
ADD INDEX `idx_movie_keywords_mid` (Movie_id);
ALTER TABLE `movie_keywords`
ADD INDEX `idx_movie_keywords_keyword` (keyword);
```

FULLTEXT Indexes

Index Name	Table	Column(s)	Purpose
ft_movie_description	movie	description	Full-text search on movie descriptions
ft_movie_director	movie_director	director_name	Full-text search on director names
ft_movie_cast	movie_cast	actor_name	Full-text search on cast/actor names
ft_movie_keywords	movie_keywords	keyword	Full-text search on movie keywords

DDL Statements:

```
ALTER TABLE `movie`
ADD FULLTEXT INDEX `ft_movie_description` (description);
ALTER TABLE `movie_director`
ADD FULLTEXT INDEX `ft_movie_director` (director_name);
ALTER TABLE `movie_cast`
ADD FULLTEXT INDEX `ft_movie_cast` (actor_name);
ALTER TABLE `movie_keywords`
ADD FULLTEXT INDEX `ft_movie_keywords` (keyword);
```

User Activity Indexes

Index Name	Table	Column(s)	Purpose
idx_favourites_user_movie	favourites	User_id, Movie_id	Composite index for favourite lookup and toggle
idx_watchlist_user_movie	watchlist	User_id, Movie_id	Composite index for watchlist lookup and toggle
idx_watch_history_user	watch_history	User_id	Speeds up per-user history fetch
idx_watch_history_movie	watch_history	Movie_id	Supports movie-side history queries
idx_watch_history_ts	watch_history	Watched_at	Speeds up ORDER BY Watched_at DESC

DDL Statements:

```
ALTER TABLE `favourites`
ADD INDEX `idx_favourites_user_movie` (User_id, Movie_id);
ALTER TABLE `watchlist`
ADD INDEX `idx_watchlist_user_movie` (User_id, Movie_id);
ALTER TABLE `watch_history`
ADD INDEX `idx_watch_history_user` (User_id);
ALTER TABLE `watch_history`
ADD INDEX `idx_watch_history_movie` (Movie_id);
```

```
ALTER TABLE `watch_history`  
ADD INDEX `idx_watch_history_ts` (Watched_at);
```

Ratings, Chat & Mood Indexes

Index Name	Table	Column(s)	Purpose
idx_ratings_user_movie	ratings	User_id, Movie_id	Composite index for upsert and per-user rating fetch
idx_ratings_movie	ratings	Movie_id	Speeds up per-movie rating aggregation
idx_ratings_posted	ratings	Posted_at	Speeds up ORDER BY Posted_at DESC in admin & movie page
idx_chatlog_user	chat_log	User_id	Speeds up per-user chat log fetch
idx_chatlog_ts	chat_log	ts	Speeds up ORDER BY ts DESC in admin chat logs
idx_mood_user	mood_session	User_id	Speeds up per-user mood session fetch

DDL Statements:

```
ALTER TABLE `ratings`  
ADD INDEX `idx_ratings_user_movie` (User_id, Movie_id);  
ALTER TABLE `ratings`  
ADD INDEX `idx_ratings_movie` (Movie_id);  
ALTER TABLE `ratings`  
ADD INDEX `idx_ratings_posted` (Posted_at);  
ALTER TABLE `chat_log`  
ADD INDEX `idx_chatlog_user` (User_id);  
ALTER TABLE `chat_log`  
ADD INDEX `idx_chatlog_ts` (ts);  
ALTER TABLE `mood_session`  
ADD INDEX `idx_mood_user` (User_id);
```

Users Table Index

Index Name	Table	Column(s)	Purpose
idx_users_email	users	email	Speeds up JOIN on users.email — used in almost every auth query

DDL Statements:

```
ALTER TABLE `users`  
ADD INDEX `idx_users_email` (email);
```

Implementation note: each DDL statement is executed only when `information_schema.STATISTICS` confirms the index does not already exist. This pattern is used in both `ensure_indexes()` (`recommendation.py`) and `_ensure_indexes_once()` (`app.py`).